Optimization Report: 2-D BCH Encoder

Rhythm Jain

***Abstract*—The abstract goes here.**

***Index Terms*—IEEE, IEEEtran, journal, LATEX, paper, template.**

I. Introduction

ERROR correction codes are used in communication & data storage applications to ensure the uniformity of data upon storage and extraction. Error Correction Codes can be divided into two categories, algebraic and iterative. Although both methods are practically useful, we will focus on algebraic codes in this paper, such as Bose–Chaudhuri-Hocquenghem (BCH) and Reed–Solomon (RS) codes. Recent developments in higher-dimensional storage applications have spurred progress in two-dimensional error-correction codes. Blahut introduced higher-dimensional finite field Fourier Transforms in [3]. A time-domain encoding approach is very computationally intensive; thus, a frequency-domain approach is adopted in this paper. This approach requires an Inverse Discrete Finite-Field Transform (IDFFFT), so the algorithm's complexity had to be reduced to achieve an efficient hardware footprint.

The encoder architecture was optimized using algebraic properties of finite fields, leveraging conjugate classes to reduce computation in [1]. The optimization was performed across different conjugate classes to reduce the number of computations at a single point. This paper focuses on reducing the number of computations across all points by removing redundant calculations and instead forwarding them to the required points. The same idea is then extended towards 3-D BCH codes.

II. Theoretical Background

This section introduces the basic mathematical concepts used in the paper for 2-D & 3-D cyclic BCH codes and spectral-domain encoding. Theoretical background on 2-D Finite Field Fourier Transform and 2-D BCH codes has been explained in [1]. Basis for 3-D spectral domain coding have been explained below :

A. 3-D Finite Field Fourier Transform

Let $[c_{i,i',i''}]$ be denote an $n \times n' \times n''$ array over *GF(q)* and suppose n, n' & n'' divide $q^m - 1$ for some integer m. Let α and β denote elements of *GF(q^m)* of orders n , n' and n'' respectively. Then, the 3-D DFFFT array $[C_{j,j',j''}]$ of size $n \times n' \times n''$ is given by (1). Similarly, the 3-D IDFFFT array $[c_{i,i',i''}]$ is given by (2)

$$C_{j,j',j''} = \sum_{i=0}^{n-1} \sum_{i'=0}^{n'-1} \sum_{i''=0}^{n''-1} \alpha^{ij} \beta^{i'j'} \gamma^{i''j''} c_{i,i',i''} \tag{1}$$

$$c_{i,i',i''} = \frac{1}{(n \bmod q)(n' \bmod q)(n'' \bmod q)} \times \sum_{j=0}^{n-1} \sum_{j'=0}^{n'-1} \sum_{j''=0}^{n''-1} \alpha^{-ij} \beta^{-i'j'} \gamma^{-i''j''} C_{j,j',j''} \tag{2}$$

Let $c_{i,i',i''}$ be the 3-D time domain array with (i, i', i'') being time-domain variables. $C_{j,j',j''}$ is called the frequency spectrum of $c_{i,i',i''}$ for the variables $(j, j'.j'')$. Here $GF(q^m)$ has to be such that $q^m = lcm(n, n', n'') + 1$ where lcm$(\cdot)$ is the least common multiple according to [3]. Therefore, $n = n' = n''$ is chosen in this paper to ensure a simple hardware architecture.

Similar to 2-D BCH code , the conjugacy constraints for 3-D Bch code are given below:

$$C^2_{j,j',j''} = C_{(2j \bmod n),(2j' \bmod n'),(2j'' \bmod n'')} \tag{3}$$

B. 3-D BCH Codes

Three-dimensional BCH codes are defined by set of arrays $c_{i,i',i''}$ whose 3-D IDFFFT (for $n = n' = n''$) is given by

$$C_{a(m(j,j')+j''),a'(m(j,j')+j'')+1,a''(m)} = 0$$

for $1 \le j \le 2t$, $1 \le j' \le 2t$ and $1 \le j'' \le 2t$. It can be proved that the minimum distance of the code with this condition will be equal to $2t + 1$ if $gcd(a, n) = 1$.Therefore, $a = 1$, $a' = 0$, and $m(k) = 0$ for $1 \le k \le 2t$ were chosen in this paper to simplify the architecture. The resultant code can be presented as :

$$C_{j,j'} = 0 \text{ for } 1 \le j \le 2t \text{ , } 1 \le j' \le 2t \text{ , } 1 \le j'' \le 2t$$

III. Encoding Procedure

In this section, we discuss the construction of our 2-D BCH code and the encoding procedure that maps input data to output data with Error Correction capability. The exact algorithm has been detailed in [1].

We construct a $n \times n$ 2-D BCH code over $GF(q^m)$ where $2^m = n - 1$. The corresponding conjugate positions of the $2t \times 2t$ region also become zeros. Information is encoded into the remaining positions. IDFFFT is used to convert this spectral information into a binary codeword. The original idea of frequency-domain encoding was presented in [3], but the exact method for splitting the message vector into spectral elements is detailed in [1].

In [1] , a 2-D BCH code with $n = 15$ and $m = 4$ is choosen along with the primitive polynomial $p(x) = x^4 + x + 1$. Using the conjugacy constraints in (**??**). We define 59 conjugate classes, of which 44 spectral zeros constitute 11 conjugate classes, represented as C-a, C-b, ... C-k. The remaining 181

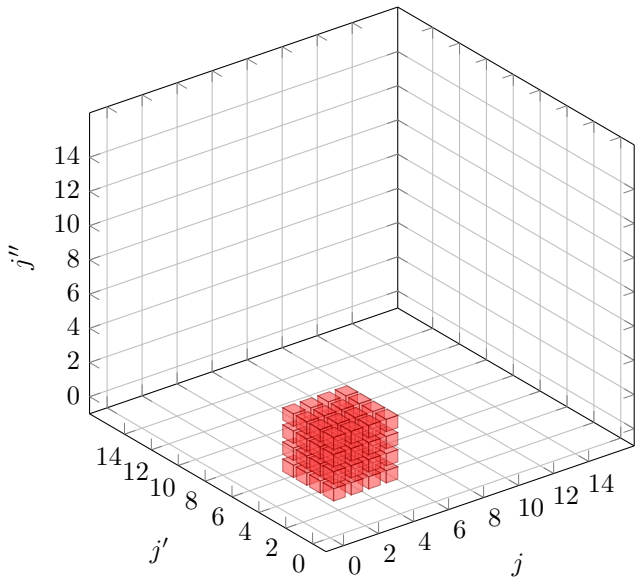


Fig. 1. Spectral nulls in a 3-D BCH code with $t = 2$

frequency positions are divided into 48 conjugate classes, represented by C-1, C-2,... C-48 . Each sub-message is mapped to an element of $GF(2^4)$ and encoded into a position known as the insertion point(IP) of the conjugate class. The remaining positions of the conjugate classes are derived from the element at the insertion point using the Frobenius condition. This native 2-D BCH code provides a rate of (181/225) for $t = 2$ error correction capability.

A. Construction of 2-D BCH Code

We have the 2-D IDFFFT from (**??**) for $n = n'$ and $q = 2$ as equations:

$$c_{i,i'} = \sum_{j=0}^{n-1}\sum_{j'=0}^{n'-1} \alpha^{-ij}\alpha^{-i'j'}C_{j,j'} = \sum_{j=0}^{n-1}\sum_{j'=0}^{n'-1} \alpha^{-(ij+i'j')}C_{j,j'} \tag{4}$$

For a certain IDFFFT point (i, i'), let $[\hat{C}_{j,j'}] = [\alpha^{ij+i'j'}]$. Now, let $[\overline{C}_{j,j'}] = [C_{j,j'}]\cdot[\hat{C}_{j,j'}]$ be the Hadamard product (i.e., point-wise multiplication of elements) of $[C_{j,j'}]$ and $[\hat{C}_{j,j'}]$. Using this, we can rewrite (4) as

$$c_{i,i'} = \sum_{j=0}^{n-1}\sum_{j'=0}^{n'-1} \overline{C}_{j,j'} \tag{5}$$

From (5), we observe that adding all elements in the n × n array $[\overline{C}_{j,j'}]$ results in the 2-D IDFFFT at a certain point (i, i'). We know that by construction, the spectrum $[C_{j,j'}]$ has a particular conjugate class structure. We show in the following theorem that the elements of $[\hat{C}_{j,j'}]$ and the Hadamard product $[\overline{C}_{j,j'}]$ are also segregated into the same conjugate class structure.

Theorem 1 : The elements of $[\hat{C}_{j,j'}]$ and the Hadamard product $[\overline{C}_{j,j'}]$ are grouped into the same conjugate class structure as $[C_{j,j'}]$.

Proof: Same as given in [1].

B. Reduction in Complexity

If $P_1(\alpha)$ & $P_2(\alpha)$ represent each element (j, j') of $[C_{j,j'}]$ & $[\hat{C}_{j,j'}]$ respectively then their product $[\overline{C}_{j,j'}]$, can be represented by $P_1(\alpha)P_2(\alpha)$. Computing the 2-D IDFFFT at any point (i, i') of $[\overline{C}_{j,j'}]$ involves computing $P_1(\alpha)P_2(\alpha)$ at all k message positions of the spectrum and adding them to get the resultant point in $[\overline{C}_{j,j'}]$.This is reduced to 1 point per conjugate class , known as the IP. Rest of the points are calculated through the conjugacy constraint i.e. by taking 2nd, 4th, 8th powers of the IP. Only the MSB of the product $P_1(\alpha)P_2(\alpha)$ is required at each IP. This is can be proved as below: Let $P_1(\alpha) = a_0 + a_1\alpha + a_2\alpha^2 + a_3\alpha^3$ and $P_2(\alpha) = b_0 + b_1\alpha + b_2\alpha^2 + b_3\alpha^3$:

$$\begin{aligned} J(\alpha) &= P_1(\alpha)P_2(\alpha) \\ &= \alpha^0(a_0b_0 + a_1b_3 + a_2b_2 + a_3b_1) \\ &+ \alpha^1\big(a_0b_1 + a_1b_0 + a_1b_3 + a_2b_2 + a_3b_1 \\ &\quad + a_2b_3 + a_3b_2\big) \\ &+ \alpha^2(a_0b_2 + a_1b_1 + a_2b_0 + a_2b_3 + a_3b_2 + a_3b_3) \\ &+ \alpha^3(a_0b_3 + a_1b_2 + a_2b_1 + a_3b_0 + a_3b_3) \\ &= j_0 + j_1\alpha + j_2\alpha^2 + j_3\alpha^3. \end{aligned}$$

The powers of J can be calculated as

$$\begin{aligned} J_2(\alpha) &= (j_0 + j_2)\alpha^0 + j_2\alpha^1 + (j_1 + j_3)\alpha^2 \\ &\quad + j_3\alpha^3, \\ J_4(\alpha) = \big(J_2(\alpha)\big)^2 &= (j_0 + j_1 + j_2 + j_3)\alpha^0 \\ &\quad + (j_1 + j_3)\alpha^1 + (j_2 + j_3)\alpha^2 + j_3\alpha^3, \\ J_8(\alpha) = \big(J_4(\alpha)\big)^2 &= (j_0 + j_1)\alpha^0 + (j_2 + j_3)\alpha^1 + j_1\alpha^2 \\ &\quad + j_3\alpha^3 \end{aligned}$$

For a conjugate class of size 4 , On adding all the powers , we get only the MSB left which is

$$\begin{aligned} \text{out}^{\text{MSB}}_{\text{size 4}} &= j_0 \oplus (j_0 \oplus j_2) \oplus (j_0 \oplus j_1 \oplus j_2 \oplus j_3) \oplus (j_0 \oplus j_1) \\ &= j_3 = a_0b_3 + a_1b_2 + a_2b_1 + a_3b_0 + a_3b_3. \end{aligned}$$

Similarly for a size 2 conjugate class , we get:

$$\begin{aligned} \text{out}^{\text{MSB}}_{\text{size 2}} &= j_0 \oplus (j_0 \oplus j_2) = j_2 \\ &= a_0b_2 + a_1b_1 + a_2b_0 + a_2b_3 + a_3b_2 + a_3b_3. \end{aligned}$$

Analyzing the total number of gates needed per point , we can see that the multiplier requires five AND gates and four XOR gates per IP for a size 4 conjugate and six AND gates and five XOR gates for size 2 conjugate class.

IV. Baseline Architecture

The architecture is described in Fig.2. The first module splits the incoming 181 bits into 48 distinct classes. Each of the conjugate classes is denoted by $S^{IP}_{cc_l}$. The next stage consists of 225 IDFFFT modules, which compute 2-D IDFFFT for all (i, i') $0 \le i, i' \le n - 1$.

The IDFFFT block is shown in Fig. 2(b). Each IDFFFT consists of 48 conjugate blocks, one for each conjugate class. All the outputs from these multipliers feed into an XOR array

TABLE I
GATE COUNT COMPARISON FOR 2-D IDFFFT AT (i, i').

Gate	Using complexity reduction	Brute force (181 message points)
AND	$4 \times 6 + 43 \times 5 = 239$	$181 \times 22 = 3982$
XOR	$4 \times 5 + 43 \times 4 + 47 = 239$	$181 \times 18 + 180 \times 4 = 3978$

for summation. The result of summation is the final output for that point (i, i').

Each of these conjugate blocks have two input, $(\alpha^{-ij-i'j'})^{IP}_{cc_l}$ and $S^{IP}_{cc_l}$ corresponding to the elements of $[\hat{C}_{j,j'}]$ and $[C_{j,j'}]$. The value of i, i', j, j' is fixed for a particular conjugate block and a particular IDFFFT block, thus they are known at the time of design in the n^2 conjugate blocks, so we can hard-code the value of $(\alpha^{-ij+i'j'})^{IP}_{cc_l}$. This paper focuses on this exact point to further optimize the encoder architecture.

Let's assume that the number of conjugate classes is l. The baseline architecture computes every IP for (i, i') for $0 \leq i \leq n, 0 \leq i' \leq n$ but the multiplication is done in $GF(q^m)$ where $q^m = n + 1$ so the number of elements in the field is $n + 1$ but the number of conjugate blocks is equal to product of number of conjugate classes and number of points. The number of multiplications is $n * l$, whereas the number of resultants can only be $n+1$, so there must be some redundancy in the calculations.

Until this point, we have been observing the encoding process from the point of view of a single point (i, i') but if change our point of view, and fix a conjugate class IP and vary the point (i, i') for $0 \leq i \leq n, 0 \leq i' \leq n$, we can observe $(\alpha^{-ij-i'j'})^{IP}_{cc+l}$ of $[\hat{C}_{j,j'}]$ repeating across the $n \times n$ grid of points.

For this example, we take $n = 15$ and organize our observations. For a conjugate class with IP (j, j'), we define a computation class as the number of distinct non-zero powers of α among the 225 values when $(i, i)' \in (0, ..., 14)$. This count is always a divisor of 225 and takes the values 15, 5, 3, or 1. The four computation classes, along with examples, are given below. Each table represents the power that has to be multiplied by the IP of a particular conjugate class.

A. 15-Computation class (C-18)

This consists of all the conjugate classes where the power of α is required 15 times each. The total number of such conjugate classes is 48, out of which 10 are spectral nulls. The exact powers of alpha to be multiplied for each output point is detailed in Table II with insertion point $(J, J') = (3, 5)$ & class size 4.

B. 5-Computation Class (C-7)

This computation class consists of 5 of the powers of α, each of which repeats 45 times among the 225 points. There are 6 such conjugate classes, out of which one is a spectral null. The exact powers of alpha to be multiplied for each output point is detailed in Table III with insertion point $(J, J') = (0, 3)$ & class size 4.

TABLE II
VALUES (POWERS OF α) GENERATED BY C-18 ACROSS 225 POINTS

$i \backslash i'$	1	2	3	4	5	6	7	8	9	10	11	12	13	14	0
1	7	2	12	7	2	12	7	2	12	7	2	12	7	2	12
2	4	14	9	4	14	9	4	14	9	4	14	9	4	14	9
3	1	11	6	1	11	6	1	11	6	1	11	6	1	11	6
4	13	8	3	13	8	3	13	8	3	13	8	3	13	8	3
5	10	5	0	10	5	0	10	5	0	10	5	0	10	5	0
6	7	2	12	7	2	12	7	2	12	7	2	12	7	2	12
7	4	14	9	4	14	9	4	14	9	4	14	9	4	14	9
8	1	11	6	1	11	6	1	11	6	1	11	6	1	11	6
9	13	8	3	13	8	3	13	8	3	13	8	3	13	8	3
10	10	5	0	10	5	0	10	5	0	10	5	0	10	5	0
11	7	2	12	7	2	12	7	2	12	7	2	12	7	2	12
12	4	14	9	4	14	9	4	14	9	4	14	9	4	14	9
13	1	11	6	1	11	6	1	11	6	1	11	6	1	11	6
14	13	8	3	13	8	3	13	8	3	13	8	3	13	8	3
0	10	5	0	10	5	0	10	5	0	10	5	0	10	5	0

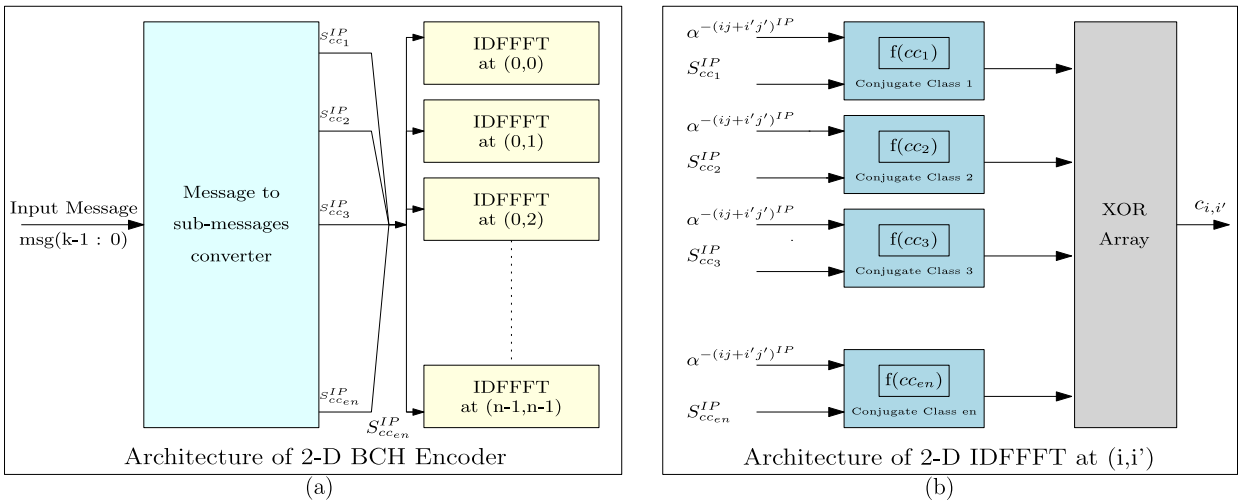


Fig. 2. (a) Architecture of encoder to compute the 2-D IDFFFT. (b) Architecture of the IDFFFT block consisting of cc_{en} parallel units to compute the 2-D IDFFFT at a certain point (i, i').

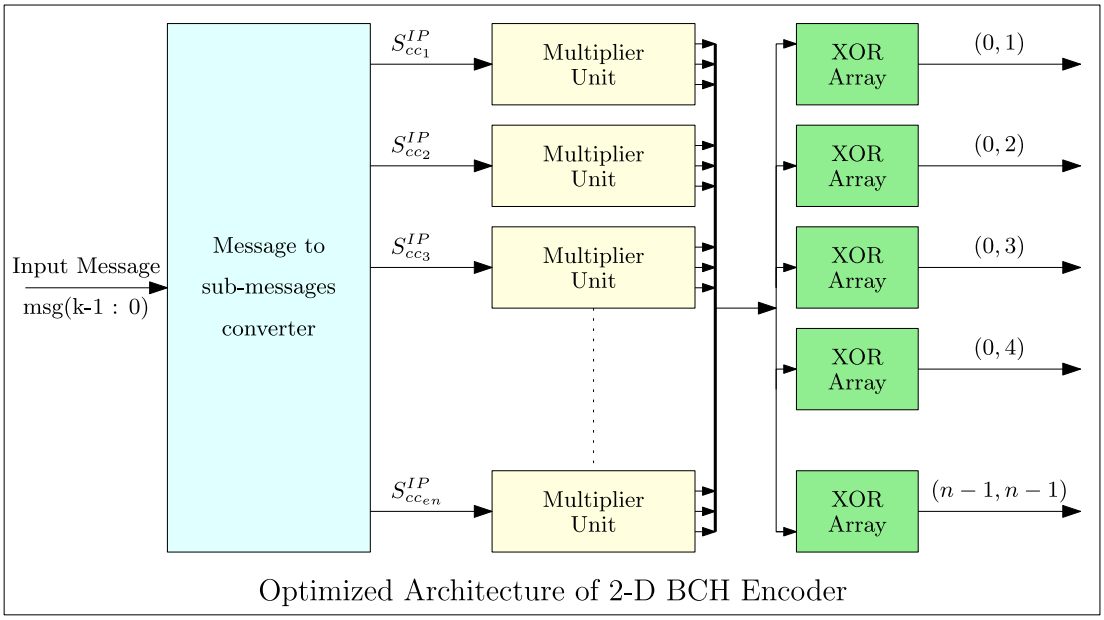


Fig. 3. Optimized Architecture of BCH 2-D Encoder

Only the powers $\{0,3,6,9,12\}$ ever appear, each 45 times ($5\times45=225$): five distinct multiplier evaluations replace 225 instances.

C. 3-Computation Class (C-3)

This class contains only 3 powers of α. This category consists of 4 conjugate classes with no spectral nulls. The exact powers of alpha to be multiplied for each output point is detailed in Table IV with insertion point $(J,J')=(5,0)$ & class size 2.

Only the powers $\{0,5,10\}$ ever appear, each 75 times ($3\times75=225$): three distinct multiplier evaluations replace 225.

D. 1-Computation Class (C-1)

The exact powers of alpha to be multiplied for each output point is α^0 with insertion point $(J,J')=(0,0)$ & class size 1. C-1 always contributes 0 i.e. only a short-circuit.

TABLE III
VALUES (POWERS OF α) GENERATED BY C-7 ACROSS 225 POINTS

$i\backslash i'$	1	2	3	4	5	6	7	8	9	10	11	12	13	14	0
1	12	9	6	3	0	12	9	6	3	0	12	9	6	3	0
2	12	9	6	3	0	12	9	6	3	0	12	9	6	3	0
3	12	9	6	3	0	12	9	6	3	0	12	9	6	3	0
4	12	9	6	3	0	12	9	6	3	0	12	9	6	3	0
5	12	9	6	3	0	12	9	6	3	0	12	9	6	3	0
6	12	9	6	3	0	12	9	6	3	0	12	9	6	3	0
7	12	9	6	3	0	12	9	6	3	0	12	9	6	3	0
8	12	9	6	3	0	12	9	6	3	0	12	9	6	3	0
9	12	9	6	3	0	12	9	6	3	0	12	9	6	3	0
10	12	9	6	3	0	12	9	6	3	0	12	9	6	3	0
11	12	9	6	3	0	12	9	6	3	0	12	9	6	3	0
12	12	9	6	3	0	12	9	6	3	0	12	9	6	3	0
13	12	9	6	3	0	12	9	6	3	0	12	9	6	3	0
14	12	9	6	3	0	12	9	6	3	0	12	9	6	3	0
0	12	9	6	3	0	12	9	6	3	0	12	9	6	3	0

TABLE IV
VALUES (POWERS OF α) FOR CLASS C-3, CONSTANT ACROSS ALL i'

i	Value
1	10
2	5
3	0
4	10
5	5
6	0
7	10
8	5
9	0
10	10
11	5
12	0
13	10
14	5
0	0

V. OPTIMIZED ARCHITECTURE

We can observe the redundant calculations in the above architecture and change it to forward the signals at the required positions rather than re-computing them. The Fig.(3) depicts the new architecture. We divide the input bit-stream into sub-messages, as in the previous design. Each sub-message corresponds to a conjugate class cc_l. Then each of them is forwarded to a multiplier unit.

Each multiplier unit is sized according to the computation class. If a conjugate class cc_l belongs to an n-computation class, then the conjugate class gives n outputs. The classification of tables is given below. The conjugate classes specified as spectral nulls are not taken as input by the message-to-sub-message module.

The multiplier unit takes a conjugate class IP as input and produces n outputs corresponding to the n powers of α with the IP. Thus, 48 multiplier units are instantiated. These signals are then routed to XOR banks for points (i,i') for $0\leq i\leq$

TABLE V
GATE COUNT SUMMARY BY COMPUTATION CLASS

Comp. class	Classes (total)	Spectral nulls	Msg.-carry. classes	AND /class	XOR /class	Total AND	Total XOR
15 Comp.	48	10	38	75	60	2850	2280
5 Comp.	6	1	5	25	20	125	100
3 Comp.	4	0	4	24	20	96	80
1 Comp.	1	0	1	0	0	0	0
Total	**59**	**11**	**48**	—	—	**3071**	**2460**

$n, 0 \leq i' \leq n$. Since the values of i, i', j & j' are known at the time of the synthesis for a particular point (i, i') and a conjugate class IP (j, j'), we can route the outputs from the multipliers to XOR adders accordingly.

The baseline architecture has 239 AND and 239 XOR gates for each of the 225 points, resulting in a total of $239 \times 225 = 53,775$ AND and $239 \times 225 = 53,755$ XOR gates. Against this baseline, the optimized architecture's 3071 AND gates and 2460 XOR gates give a reduction of

$$\begin{aligned} \frac{53,775}{3071} &= 17.51 \times fewer\ AND\ gates \\ \frac{53,775}{2460} &= 21.86 \times fewer\ XOR\ gates \end{aligned} \tag{6}$$

This optimizes only the multiplication part of the encoder, but a large share of the hardware cost is due to the XOR trees, which remain unchanged from the previous design.

A. Construction of 3-D BCH Code

We define a $n \times n \times n$ 3-D BCH code over $GF(q^m)$ where $2^m = n - 1$. We choose the primitive polynomial as $p(x) = x^4 + x + 1$ along with $n = 15$ & $m = 4$ .The corresponding positions of the $2t \times 2t \times 2t$ region , along with their respective conjugate classes become zeroes. Information is encoded in to the remaining conjugate classes using the same process as used in the 2-D BCH code i.e. splitting the input vector into spectral elements. The conjugate class structure with 851 classes has been described in Table VI. It can be observed that we get a total of 3163 message points out of 3375 points which gives us a code rate of $3163/3375 = 0.937$.

B. Reduction of Algorithmic Complexity

We have 3-D IDFFFT from 2 for $n = n' = n''$ and $q = 2$ as

$$\begin{aligned} c_{i,i',i''} &= \sum_{j=0}^{n-1} \sum_{j'=0}^{n'-1} \sum_{j''=0}^{n''-1} \alpha^{-ij} \alpha^{-i'j'} \alpha^{-i''j''}\, C_{j,j',j''} \\ &= \sum_{j=0}^{n-1} \sum_{j'=0}^{n'-1} \sum_{j''=0}^{n''-1} \alpha^{(-ij-i'j'-i''j'')}\, C_{j,j',j''} \end{aligned} \tag{7}$$

For a certain IDFFFT point (i, i', i''), let $[\hat{C}_{j,j',j''}] = [\alpha^{ij+i'j'+i''j''}]$. Now, let $[\overline{C}_{j,j',j''}] = [C_{j,j',j''}] \cdot [\hat{C}_{j,j',j''}]$ be the Hadamard product $[C_{j,j',j''}]$ and $[\hat{C}_{j,j',j''}]$. Using this, we can rewrite (7) as

$$c_{i,i',i''} = \sum_{j=0}^{n-1} \sum_{j'=0}^{n'-1} \sum_{j''=0}^{n''-1} \overline{C}_{j,j'} \tag{8}$$

Theorem 2 : The elements of $[\hat{C}_{j,j',j''}]$ and the Hadamard product $[\overline{C}_{j,j',j''}]$ are grouped into the same conjugate class structure as $[C_{j,j',j'''}]$.

Proof : Let the α^k be the element of $[C_{j,j',j''}]$ at point (j, j', j'') , then the it's conjugate class will be $[\alpha^k, \alpha^{2k}, \alpha^{3k}, ..., \alpha^{2^{m-1}k}]$ on points $[(j, j', j''), (2j, 2j', 2j''), ..., (2^{m-1}j, 2^{m-1}j', 2^{m-1}j'')]$. The respective elements of $[\hat{C}_{j,j',j''}]$ are $[\alpha^{-(i.j+i'.j'+i''.j'')}, \alpha^{-(i.2j+i'.2j'+i''.2j'')}$, ..., $\alpha^{-(i.2^{m-1}j+i'.2^{m-1}j'+i''.2^{m-1}j'')}]$. Rewriting the elements , we get $[\alpha^{-(i.j+i'.j'+i''.j'')}, \alpha^{-2(ij+i'j'+i''j'')}$, ... , $\alpha^{-2^{m-1}(ij+i'j'+i''j'')}]$

C. Results

The encoder was synthesized and verified on Vivado Xilinx for an Artix-7 FPGA. The baseline architecture and the optimized version were simulated and tested for the same input data. Vivado minimizes the Boolean equations across the Multiplier Units and XOR trees using 6-input LUTs. The baseline architecture used 1898 LUTs, whereas the optimized version achieved a final footprint of 1648 LUTs, resulting in a 13.2% reduction in hardware footprint.

VI. CONCLUSION

The conclusion goes here.

APPENDIX A
PROOF OF THE FIRST ZONKLAR EQUATION

Appendix one text goes here.

APPENDIX B

Appendix two text goes here.

ACKNOWLEDGMENT

The authors would like to thank...

REFERENCES

[1] A. Mondal and S. S. Garani, "Efficient Hardware Architectures for 2-D BCH Codes in the Frequency Domain for Two-Dimensional Data Storage Applications," *IEEE Transactions on Magnetics*, vol. 57, no. 5, pp. 1-14, May 2021, doi: 10.1109/TMAG.2021.3060807.

[2] H. Imai, "A theory of two-dimensional cyclic codes," Inf. Control, vol. 34, no. 1, pp. 1–21, May 1977.

[3] R. E. Blahut, "Transform techniques for error control codes," IBM J. Res. Develop., vol. 23, no. 3, pp. 299–315, May 1979.

TABLE VI
3-D BCH CONJUGATE CLASS STRUCTURE

Class size	Points	Classes	Spectral Nulls	Message Points
1	1 (0,0,0)	1	0	1
2	26	13	0	26
4	3348	837	53	3136
			Total	3163



Michael Shell Biography text here.

John Doe Biography text here.

Jane Doe Biography text here.